CSL251: Computer Organization and Architecture
Scalar Pipelined Processor Design with Hazard Handling
Lab-10 Problem Statement

1 Instructions

- This assignment is to be done in a group of maximum 2 members.
- Code must be written in C++.
- Submission by both the members is not necessary, any one of the two members of the group can submit with the file name as mentioned below.
- Submit a zip file following the directory structure given below:

```
<ROLLNO1>_<ROLLNO2>_LAB10/
    input/
        ICache.txt
        DCache.txt
        RF.txt
    output/
        DCache.txt
        Output.txt
```

- Deadline: 23/03/2026 11:59PM

2 Objective

- To understand the working of a scalar pipelined processor by simulating different components.

3 Processor Configuration

Consider a scalar pipelined processor with:

- 256B Instruction Cache (I$)
- 256B Data Cache (D$)

- Both are direct-mapped and perfect (no misses)
- Register File (RF) with 16 registers (R0–R15)
- R0 always stores 0
- Two read ports and one write port
- Negative numbers stored in 2's complement

4 Instruction Set

4.1 Arithmetic Instructions

- ADD rd rs1 rs2 : $rd \leftarrow rs1 + rs2$
- SUB rd rs1 rs2 : $rd \leftarrow rs1 - rs2$
- MUL rd rs1 rs2 : $rd \leftarrow rs1 * rs2$
- INC r : $r \leftarrow r + 1$

4.2 Logical Instructions

- AND rd rs1 rs2 : $rd \leftarrow rs1 \& rs2$
- OR rd rs1 rs2 : $rd \leftarrow rs1 | rs2$
- XOR rd rs1 rs2 : $rd \leftarrow rs1 \oplus rs2$
- NOT rd rs : $rd \leftarrow \sim rs$

4.3 Shift Instructions

- SLLI rd rs1 imm(4) : $rd \leftarrow rs1 << imm$
- SRLI rd rs1 imm(4) : $rd \leftarrow rs1 >> imm$

4.4 Memory Instructions

- LI rd imm(8) : $rd \leftarrow imm$
- LD rd rs1 imm(4) : $rd \leftarrow [rs1 + imm]$
- ST rd rs1 imm(4) : $[rs1 + imm] \leftarrow rd$

4.5 Control Instructions

- JMP L1 : Jump by L1 instructions
- BEQZ rs L1 : Jump if rs == 0

4.6 Halt

- HLT : Terminate program

5 Opcode Encoding

Opcode	Encoding	Opcode	Encoding
ADD	0000	SLLI	1000
SUB	0001	SRLI	1001
MUL	0010	LI	1010
INC	0011	LD	1011
AND	0100	ST	1100
OR	0101	JMP	1101
XOR	0110	BEQZ	1110
NOT	0111	HLT	1111

6 Pipeline

Five stages:

$$\text{IF} \rightarrow \text{ID} \rightarrow \text{EX} \rightarrow \text{MEM} \rightarrow \text{WB}$$

6.1 IF

```
IR ← Icahe[PC]
PC ← PC + 2
```

6.2 ID

Decode instruction and read registers:

```
A ← RF[rs1]
B ← RF[rs2]
For STORE  inst B=RF[rd] {CONSIDER format of store ST rd,rs1,offset}
```

6.3 EX

- ALU: $ALUOutput = A\ op\ B$
- Memory: $ALUOutput = A + imm$
- Branch: $ALUOutput = PC + L1 \times 2$

6.4 MEM

- LOAD: $LMD = Dcache[ALUOutput]$
- STORE: $Dcache[ALUOutput] = B$

6.5 WB

- $RF[rd] = ALUOutput$
- $RF[rd] = LMD$

7 Hazards

7.1 RAW Hazard

Example:

```
AND R1 R2 R3
SUB R4 R1 R5
```

7.2 Control Hazard

- Occurs due to JMP/BEQZ
- Pipeline is stalled for 2 cycles

8 Input Format

- ICache.txt (256 lines)
- DCache.txt (256 lines)
- RF.txt (16 lines)

Note:

- Students must implement **all the above instructions** based on the opcode encoding table.
- Each instruction is 16-bit and must be decoded according to opcode and operand fields.
- Proper handling of RAW hazards and control hazards (JMP, BEQZ) must be implemented.

9 Output Format

- Updated DCache.txt
- Output.txt containing:
  - Total instructions
  - Instruction types
  - CPI
  - Number of stalls